

DDalphaAMG 的多重网格算法实现

源码取证、数学结构与当前快照边界

源码分析（物理 + 代码视角）

DDalphaAMG Wilson-Clover 求解器快照

2026-08-28

核心判断：算法链清晰，但当前快照不足以声称端到端可复现

设计层 确证

用户手册明确将问题定义为 Wilson-Dirac 方程的聚合型 AMG；SAP 是平滑器，K-cycle 和混合精度由参数控制。

实现层 确证

当前树保留 level 状态、插值/限制接口、Galerkin 粗算子块乘、CUDA 细格 Dirac 与奇偶 Schur 组件。

边界 未验证

setup/V-cycle/interpolation 的 CPU 主体不在当前文件树；CUDA 粗层 Schur 有明确 TODO，未有本次运行数据。

本报告的结论口径：把源码中可以逐行核验的算子/数据布局，与由接口、参数和用户手册共同支持的算法意图分开；**不把历史报告中指向缺失 '.c' 文件的行号当作当前实现证据。**

来源：设计：doc/user_doc.tex:21--26,79--85；状态：src/level_struct.h:13--90、src/coarse_operator_generic.h:29--57、src/gpu/cuda_coarse_oddeven_generic.cu:24--91

问题不是“把矩阵变小”，而是把低模误差交给可表示的粗空间

$$D_W(U, m_0) \psi = \eta,$$
$$R_\ell = P_\ell^\dagger,$$

$$m_0 = \frac{1}{2\kappa} - 4,$$
$$D_{\ell+1} = R_\ell D_\ell P_\ell.$$

对象	物理/数值含义	报告中的验收口径
U, m_0, c_{sw}	规范背景、质量和 clover 参数；细格算子作用在每格点 $12 = 4 \times 3$ 个复自由度上。	只确认输入约定，不从参数样例推导性能。
P	聚合内 test vectors 张成的局部低模近似空间。	必须能解释聚合、局部正交化和粗格自由度。
S_ℓ	SAP/局部求解器，优先消除粗化前仍然振荡的误差。	必须说明块、着色、局部迭代和 odd-even 分支。
$D_{\ell+1}$	Galerkin 粗算子，负责低频/近零模误差的校正。	必须给出 $P^\dagger D P$ 、存储块和作用路径。

极限检查

$\kappa \rightarrow 0$ 时质量 shift 主导、系统趋于对角； $c_{sw} \rightarrow 0$ 时 clover 分支退化。它们是理解算子边界的检查，不是当前快照的运行结果。

来源：方程/参数：doc/user_doc.tex:21--26,76--85；细格 CUDA 作用：src/gpu/cuda_dirac_generic.cu:309--410；物理极限为公式推断。

层级是显式对象：每个 level 同时携带算子、粗化、平滑与求解状态

level 状态	代码字段	在多重网格中的职责
细/粗算子	op_double/op_float、 oe_op_double/oe_op_float	原始 Dirac/粗算子与 odd-even 版本；精度实例并存。
插值层	is_double/is_float	聚合数、索引、test_vector、interpolation、bootstrap 向量与粗算子数据。
平滑层	s_double/s_float	块列表、颜色、边界、局部缓冲与块内求解所需状态。
Krylov 层 几何/容量	p_*、sp_*、dummy_p_* global_lattice、local_ lattice、coarsening、 vector_size	K-cycle、平滑器和占位求解器的 GMRES 状态。记录本层格点、聚合比、局部/ghost 体积与自由度。
递归关系	next_level、depth	从当前 level 进入更粗 level；最粗层以无下一级为边界。

输入($U, \eta, geometry$) $\longrightarrow$ level 初始化 $\longrightarrow$ setup / $P, D_c \longrightarrow$ 外层 FGMRES 调用多重网格 $\longrightarrow \psi$
流程接口包括 method_setup、next_level_setup 与 method_update。

来源：结构：src/level_struct.h:13--90；插值结构：src/algorithm_structs_generic.h:271--314；流程接口：src/init.h:29--44。

几何聚合把细格局部自由度压缩为粗格的 $2n_\ell$ 个分量

$$a_\ell(\mu) = \frac{L_\ell(\mu)}{L_{\ell+1}(\mu)},$$

$$n_\ell = \text{本层 test vectors 数},$$

$$N_{\ell+1}^{\text{site}} = 2n_\ell \quad (\text{粗层自旋块约定}).$$

约束/样例	当前仓库证据	对实现的影响
聚合整除	$L_\ell/L_{\ell+1}$ 、局部格点/聚合和聚合/块均要求为正整数。	聚合索引可以预计算；跨聚合 hop 落在边界表中。
进程收缩	全局/局部格点的进程比例随层变粗单调不增。	粗层允许进程 idle，level 通信子不能简单等同于细层。
样例 $d0 \rightarrow d1$	$8^4 \rightarrow 2^4$ ，故 $a_0(\mu) = 4$ ； $d0$ 块为 4^4 ，有 24 个 test vectors。	这是可复现的配置样例，不是性能基线。
最粗层	样例 $d1$ 的 test/setup 参数为 0。	粗层不再建立下一级 AMG；转交粗网格求解接口。

fine site (12 complex d.o.f.)

$\xrightarrow[\text{aggregate}]{P^\dagger}$

coarse site ($2n_\ell$ complex d.o.f.)

该映射由 level、插值和粗算子索引支持；聚合构造函数体缺失。

来源：几何约束：doc/user_doc.tex:52--71；样例：sample_devel.ini:34--55；字段：src/level_struct.h:51--73、src/algorithm_structs_generic.h:308--314。

自适应 setup 的核心是“平滑试验向量—局部正交—重建粗空间”闭环

Input: $V^{(0)}$, $\{a_\ell, n_\ell\}$, setup_iter
Initialize: allocate test_vector, interpolation, bootstrap_vector
for $k = 0, 1, \dots, \text{setup_iter} - 1$ iterate:
 orthogonalize $V^{(k)}$ on each aggregate
 relax/inverse-iterate $V^{(k)}$ with the current multilevel preconditioner
 define $P^{(k+1)}$ from the aggregate-local vectors
 rebuild $D_{\ell+1}^{(k+1)} = (P^{(k+1)})^\dagger D_\ell P^{(k+1)}$
 if the next level exists, bootstrap/setup it; otherwise stop setup recursion
Output: P_ℓ , $D_{\ell+1}$, level-local solver state

证据等级	解释
确证	接口直接暴露 iterative_setup、re_setup、interpolation_define、inv_iter_inv_fcycle; 线性代数接口直接暴露 aggregate Gram-Schmidt。
推断	上表的“平滑 → 重建 → 递归”是由接口命名、test/bootstrap 字段与用户手册的 interpolation 模式拼出的算法契约。
未验证	当前文件树没有 setup/interpolation/vcycle 的函数体, 故无法确认每个版本的精确调用顺序、停止判据和数值收益。

来源: setup 接口: src/setup_generic.h:27--32; Gram-Schmidt 接口: src/linalg_generic.h:122--140; 状态字段: src/algorithm_structs_generic.h:308--314; 模式: sample_devel.ini:91--100。

限制、粗算子与延拓组成一个可检查的 Galerkin 粗校正

Input: x_ℓ, b_ℓ, P_ℓ
 $r_\ell \leftarrow b_\ell - D_\ell x_\ell$
 $r_{\ell+1} \leftarrow R_\ell r_\ell, \quad R_\ell = P_\ell^\dagger$
 $D_{\ell+1} \leftarrow R_\ell D_\ell P_\ell$
 $e_{\ell+1} \leftarrow \mathcal{M}_{\ell+1}(r_{\ell+1})$
 $x_\ell \leftarrow x_\ell + P_\ell e_{\ell+1}$
Output: corrected x_ℓ and residual for post-smoothing

操作	当前接口	必须保持的数学关系
延拓 限制	interpolate、interpolate3 restrict	粗向量回到细格；其局部支撑由聚合布局决定。在复数内积约定下对应 $P^\dagger$ ，不能误当作普通注入。
粗算子 setup	coarse_operator_setup、 set_coarse_*_coupling	逐聚合 self/neighbor coupling 组织 $P^\dagger DP$ 。
粗算子 apply	apply_coarse_operator、apply_ coarse_operator_dagger	递归求解和伴随/对称性检查共用粗层表示。

正确性检查

若 P 的列空间确实承载近零模，粗校正应主要消除平滑器难以处理的低频误差；这是 AMG 的数值设计目标，当前快照没有提供收敛曲线来验证它。

来源：接口：src/interpolation_generic.h:27--36；粗算子接口：src/coarse_operator_generic.h:29--57；
 $D_c = P^\dagger DP$ 为 Galerkin 定义。

粗算子存储显式利用块结构: Hermitian 对角块 + 一个非对角块

$$D_c(x, \mu) = \begin{pmatrix} A & B \\ C & D \end{pmatrix},$$

$$A = A^\dagger, \quad D = D^\dagger, \quad C = -B^\dagger,$$

$$D_c(x, -\mu) = \begin{pmatrix} A^* & -C^* \\ -B^* & D^* \end{pmatrix},$$

$$n = \frac{\text{num_lattice_site_var}}{2}.$$

组件	存储/作用方式	代码证据与含义
普通 $D\phi$	mv/nmv 对列存 $n \times n$ 块乘; 四块按 A, C, B, D 依次移动指针。	直接展示粗 hop 的块布局 and 符号。
伴随 $D^\dagger \phi$	mvh/nmvh 使用共轭转置; 负号编码 $C = -B^\dagger$ 。	不需要独立复制全部方向块。
Hermitian self block	mvp 只读取 packed 上三角并补出下三角。	代码注释明确 self coupling 的 A, D 为 Hermitian。
SIMD 路径	SSE 版本将 self/neighbor coupling 按 spin 块和列方向组织, 并提供 vectorized copy。	这是布局优化证据; 不等同于完整 CUDA AMG。

保存 $\{\text{upper}(A), \text{upper}(D), B\} \implies$ 作用时由共轭转置恢复 C

推断 相对完整 $2n \times 2n$ 矩阵可降低存储/算术量; 实际收益需 benchmark, 当前未验证。

来源: 块结构/列存乘: src/coarse_operator_generic.h:62--117, 119--225; SSE 映射:

src/sse_coarse_operator.h:42--139, 142--240。

V-cycle 是递归骨架，K-cycle 用 FGMRES 重新组合两层预条件器

Input: (ϕ_ℓ, η_ℓ) , D_ℓ , P_ℓ , level ℓ
pre-smooth: $\phi_\ell \leftarrow S_\ell(\phi_\ell, \eta_\ell)$
 $r_\ell \leftarrow \eta_\ell - D_\ell \phi_\ell$
if $\ell = \ell_{\text{coarse}}$: $e_\ell \leftarrow \text{coarse solver}(r_\ell)$
else: $r_{\ell+1} \leftarrow P_\ell^\dagger r_\ell$; $e_{\ell+1} \leftarrow \mathcal{M}_{\ell+1}(r_{\ell+1})$; $e_\ell \leftarrow P_\ell e_{\ell+1}$
 $\phi_\ell \leftarrow \phi_\ell + e_\ell$
post-smooth: $\phi_\ell \leftarrow S_\ell(\phi_\ell, \eta_\ell)$; Output: ϕ_ℓ

模式	用户可调参数	算法含义
V-cycle	d_i preconditioner cycles、post smooth iter	递归一次粗校正，再回到细格后平滑。
K-cycle	kcycle=1、length、restarts、tolerance	用户手册将粗网格 GMRES 替换为由下一层两水平法预条件的 FGMRES；可视作带 FGMRES 包装的 W 型递归。
最粗层	coarse_solve_odd_even、coarse_apply_schur_complement	粗层不再递归，转为 GMRES/odd-even 相关求解接口。
当前证据边界	src/vcycle_generic.h 只有声明与依赖	循环契约可确认；V/K 调度函数体不在当前文件树，具体实现仍需补齐。

来源：V-cycle 接口：src/vcycle_generic.h:25--39；最粗层接口：src/coarse_oddeven_generic.h:30--45；
K-cycle 语义/参数：doc/user_doc.tex:79--85、sample_devel.ini:102--109。

SAP 平滑器处理局部高频误差， odd-even Schur 把块内代数结构显式化

$$D = \begin{pmatrix} D_{ee} & D_{eo} \\ D_{oe} & D_{oo} \end{pmatrix},$$

$$S_e = D_{ee} - D_{eo}D_{oo}^{-1}D_{oe}.$$

平滑分支	接口/参数	作用
Additive Schwarz	additive_schwarz; method 1	各块独立更新，适合并行但不显式串行传播最新边界。
Red-black Schwarz	red_black_schwarz; method 2	两色块更新，依赖关系可控；样例测试大量采用该路径。
16-color Schwarz	sixteen_color_schwarz; method 3	更细粒度着色；同色块无直接边界依赖。
块内求解	block_solve_oddeven、local_minres; blockiter	用 Schur 约化或局部最小残量迭代完成块内近似逆。

SAP step: $r \leftarrow \eta - D\phi \longrightarrow$ select color/block $\longrightarrow$ local solve $\longrightarrow \phi \leftarrow \phi + \omega \delta\phi$

高频误差由块内局部求解衰减；低频误差留给 $P^\dagger DP$ 的粗校正。

来源：平滑器接口：src/schwarz_generic.h:36--61; odd-even/块求解接口：src/oddeven_generic.h:39--61; method 参数：sample_devel.ini:118--128。当前树无对应 CPU 函数体。

CUDA 奇偶路径可逐步核验：投影/通信/对角逆/Schur 组合已落地

```
Input:  $u = (u_e, u_o)$   
 $y_e \leftarrow D_{ee} u_e$   
 $t_o \leftarrow D_{oe} u_e$  (odd target: hopping + ghost exchange)  
 $q_o \leftarrow D_{oo}^{-1} t_o$   
 $t_e \leftarrow D_{eo} q_o$   
 $y_e \leftarrow y_e - t_e = (D_{ee} - D_{eo} D_{oo}^{-1} D_{oe}) u_e$   
Output:  $y_e$  in componentwise/chunkwise wrapper
```

源码阶段	直接观察	实现含义
D_{ee}	cuda_diag_ee_ componentwise	逐点 clover/self coupling。
D_{oe}/D_{eo}	cuda_hopping_term	只接受 _EVEN_SITES 或 _ODD_SITES；投影后进行邻居矩阵乘。
通信	cuda_ghost_sendrecv/wait	负/正方向投影分开发送和等待，允许把本地 kernel 与 halo 路径编排。
组合	cuda_apply_schur_ complement	代码顺序与 Schur 公式逐项一致，最后执行向量减法。

CUDA 后端的“细格算子已实现”不能外推为“多级 AMG 已完整实现”

观察	证据	对端到端结论的限制
细格 operator allocation	cuda_operator_alloc	在
细格 vector wrapper	l->depth!=0 时直接 return。 wrapper 在 depth 非 0 时报错，并 写明粗格内存交互尚未充分测试。	当前代码明确优先支持 depth 0 的 GPU operator 状态。 不能据此声称 CUDA 粗格延拓/限制和递 归已验证。
粗层 Schur	coarse_apply_schur_ complement_CUDA	CUDA 粗层路径是部分实现/回退混合，不 是纯 GPU 闭环。
CUDA Krylov/smoother	TODO；中间把数据拷回 CPU，调 用 CPU 粗层对角/跳跃操作。 cuda_solve_oddeven 对 GMRES smoother 和 method 5 打印 “has not been constructed”。	K-cycle 所需的 GPU FGMRES 组合不能从 当前文件得到完整确证。

严格结论

确证：CUDA 细格 Wilson–Clover、组件化布局和细格 odd-even kernel 存在。
未验证：当前快照的 CUDA 多层 setup、粗格递归和完整 K-cycle 端到端执行；性能、收敛和加速比均未测量。

来源：分配边界：src/gpu/cuda_operator_generic.cu:44--75；细格 wrapper：
src/gpu/cuda_dirac_generic.cu:631--661；粗 Schur TODO/CPU 混合：
src/gpu/cuda_coarse_oddeven_generic.cu:24--91；smoother 限制：
src/gpu/cuda_oddeven_generic.cu:4429--4451。

参数文件把算法选择暴露为可复现实验矩阵，但不提供结果本身

参数组	样例值	控制的算法行为
层级/聚合	2 层; $8^4 \rightarrow 2^4$; $d0$ block $= 4^4$	细/粗格几何、Schwarz 块和聚合比。
setup	$d0$ test vectors = 24; setup iter = 4; interpolation = 2	试验向量数量、setup 轮数和 f-cycle inverse-iteration 模式。
precision	mixed precision = 1	用户手册定义为单精度预条件器、双精度外层 FGMRES。
odd-even	= 1	粗层 GMRES/块内求解可走 odd-even; 粗层几何需满足偶性约束。
coarse solve K-cycle	30 iter; 50 restarts; 10^{-2} on; length 5; 2 restarts; 10^{-1}	最粗层 Krylov 停止与重启上限。用 FGMRES 包装两层预条件器的深度/精度。
outer solve	method 2; relative tol 10^{-10}	red-black Schwarz 外层路径与相对残差目标。

$$np = \prod_{\mu=1}^4 \frac{L_0(\mu)}{L_0^{local}(\mu)}, \quad \frac{L_\ell}{L_{\ell+1}}, \frac{L_\ell^{local}}{a_\ell}, \frac{a_\ell}{L_\ell^{block}} \in \mathbb{Z}_{>0}$$

上述是几何可用性约束；“可用”不等于“本快照已成功构建并收敛”。

来源：样例：sample_devel.ini:34--55,89--109,118--135；约束/混合精度/K-cycle: doc/user_doc.tex:52--85。

证据账本：最强结论是算法接口与局部 kernel，最大风险是缺少调度主体

主张	状态	依据/缺口
聚合型 AMG + SAP	确证	用户手册定义；setup、interpolation、Schwarz 接口成组出现。
自适应 setup 的精确循环	推断	test/bootstrap 字段、Gram–Schmidt 和 setup API 支持；函数体缺失。
$D_c = P^\dagger DP$ 的块作用	确证	粗算子接口、列存矩阵乘和 Hermitian block 注释直接可查。
V/K-cycle 的完整递归调度	推断	vcycle/K-cycle 接口与参数语义存在；当前树无主体实现。
CUDA 细格 Schur	确证	CUDA 函数顺序与 Schur 公式逐项对应。
收敛率、加速比、可扩展性	未验证	当前没有本次运行日志；不能用文献或样例参数代替测量。

下一验证门槛	可验收产物
补齐源码	找回与 generic 头匹配的 setup/interpolation/vcycle/linsolve CPU 实现及 Makefile/构建版本。
构建回归	在 $4^4/8^4$ 配置上记录 TRACK_RES、COARSE_RES、SCHWARZ_RES 和真实残差。
GPU 边界测试	单独验证 depth 0、粗层 Schur、数据回拷和 K-cycle；失败时保留最小复现日志。

来源：证据索引见附录；诊断开关：doc/user_doc.tex:94--106；测试入口：test/integration_test.sh:29--50。时间节点和性能目标均未由仓库指定。

结论：DDalphaAMG 的多重网格思想已形成闭环，当前交付应标为“部分可核验”

物理问题: $D_W(U, m_0)\psi = \eta$

↓ 近零模/低频误差需要粗空间承载

setup: test vectors $\longrightarrow$ aggregate-local basis $\longrightarrow P$

Galerkin: $D_{\ell+1} = P_{\ell}^{\dagger} D_{\ell} P_{\ell}$

solve: SAP (高频) + V/K-cycle coarse correction (低频)

implementation: level state + packed coarse blocks + odd-even/CUDA kernels

verification: local algebra is inspectable; end-to-end hierarchy and performance remain open

一句话带走

该快照最有价值的实现决策是：用自适应聚合空间表示近零模，用 Galerkin 粗算子传递低频误差，用 SAP/Schur 处理局部高频结构；但缺失的层级调度主体和 CUDA 粗层 TODO 使“完整运行实现”仍不能被当前证据闭合。

来源：综合依据：src/setup_generic.h:27--32、src/interpolation_generic.h:27--36、src/coarse_operator_generic.h:29--57、src/vcycle_generic.h:35--39、src/oddeven_generic.h:27--61。

附录 A：源码证据索引（当前快照）

主题	文件: 行号	用途
level 生命周期/容量	src/level_struct.h:13--90	细/粗算子、插值、Schwarz、GMRES、next level。
setup 契约	src/setup_generic.h:27--32	setup、重建、inverse-iteration/f-cycle 接口。
聚合/插值契约	src/coarsening_generic.h:25--29; src/interpolation_generic.h: 27--36	索引表、限制、延拓和插值算子定义入口。
粗算子局部代数	src/coarse_operator_generic.h: 62--225	列存矩阵乘、伴随乘、 $2n \times 2n$ 块结构。
粗算子 CUDA self block	src/gpu/cuda_coarse_operator_ generic.cu:24--126	shared memory、packed Hermitian self coupling kernel。
平滑器/块求解	src/schwarz_generic.h:36--61; src/oddeven_generic.h:39--61	SAP 变体、块 odd-even、局部 MINRES 接口。
cycle/linear solve	src/vcycle_generic.h:35--39; src/linsolve_generic.h:30--51	V-cycle、FGMRES/FGCR/粗层求解接口。
CUDA Schur	src/gpu/cuda_oddeven_generic.cu: 4001--4209	细格 Schur、hopping、ghost 和布局转换。
CUDA 缺口	src/gpu/cuda_coarse_oddeven_ generic.cu:24--91	粗 Schur TODO 与 CPU 混合路径。

来源：表内文件路径均相对于当前工作目录。

附录 B：接口—参数—验证命令的最小闭环

API input: `dd_alpha_amg_init (U, m0, csw, amg_params)`
↓ `dd_alpha_amg_setup(iterations, status)`
Setup state: `mg_setup_status` → P, D_c , level hierarchy
↓ `dd_alpha_amg_wilson_solve(out, in, tol, ...)`
Verification: true residual + coarse/SAP residual + no CUDA fallback ambiguity

证据	作用
<code>src/dd_alpha_amg.h:41--83</code>	外部初始化、场更新通知、setup、solve、preconditioner 和 free 的接口。
<code>src/dd_alpha_amg_parameters.h:25--50</code>	层数、几何、basis/setup/post-smooth、粗层迭代和质量参数的结构化配置。
<code>doc/user_doc.tex:94--106</code>	建议打开真实残差、粗层残差、Schwarz 残差和 test-vector 分析的诊断开关。
<code>README:21--36</code>	说明求解器定位与构建/运行入口；当前快照仍需检查实际 Makefile/CPU 实现是否恢复。

来源：本附录只列出可复查入口，不把未执行的命令或预期输出写成实测结果。